

Laravel Interview Revision Guide

Senior Backend Laravel Developer — Marketplace

PREPARED FOR
Interview Preparation

BY
Chris Garlick

DATE
2026-07-01

About This Guide

A focused Laravel refresher for the Senior Backend Laravel Developer role — a fast-growing, remote-first marketplace app used by millions, where the job is to help modernise their Laravel application. Each section opens with why it matters for this role, gives the concept in plain English, then a Laravel/PHP example you can speak to.

LARAVEL CORE

1. The Role, Mapped
2. Request Lifecycle
3. Service Container & DI
4. Providers & Facades
5. Routing & Middleware
6. Validation
7. API Resources

ELOQUENT & DATA

8. Eloquent Relationships
9. N+1 & Eager Loading
10. Scopes, Casts, Observers
11. Indexes & Table Design
12. EXPLAIN & Tuning
13. Migrations at Scale
14. Big Data in Laravel

SCALE, APIs & OPS

15. Transactions & Locking
16. Caching / Redis
17. Queues & Horizon
18. Events & Listeners
19. Stripe / AWS / Google / OpenAI
20. Webhooks & Idempotency
21. Microservices
22. Testing
23. Security
24. DevOps & Backups
25. Modernising a Legacy App

How to use it: read every Why it matters box, rehearse the Say this lines out loud, and be ready to tie each topic back to their app — a high-traffic marketplace with 800+ models, heavy data, and lots of API integrations. The last page has questions to ask them.

1 · The Role, Mapped to You

WHY IT MATTERS

Before the tech, get the story straight. They want a problem-solver, not a task-worker; someone who treats the craft as a passion and wants to grow with the company. Map every must-have to a concrete thing you've built — vague claims lose to specific examples.

Their must-haves → your evidence

They want	What to say / show
2+ yrs Laravel	A Laravel app you shipped — models, migrations, queues, the lot.
5+ yrs frameworked langs	Total framework experience across PHP/Laravel and others.
MySQL indexes & table design	A slow query you fixed with an index; a schema you designed.
Big data, and enjoys it	Largest table/dataset you've handled; how you kept it fast.
Many APIs (Stripe/Google/AWS/OpenAI)	An integration you built end-to-end, incl. webhooks.
Caching & job queues	Work you pushed to a queue; a cache that cut load.
Problem-solver mindset	The 'Google Maps 100x' type story — see §25.

THE TELL

They ask lots of intelligent questions and want the same back. Curiosity about their app — its scale, its pain points, its 800+ models — reads as genuine interest, which they explicitly value.

SAY THIS

I like owning a problem end to end — understanding the why, not just closing the ticket. On a marketplace at your scale, the interesting work is keeping it fast and reliable as the data grows.

2 · The Laravel Request Lifecycle

WHY IT MATTERS

“What actually happens when a request hits a Laravel app?” is a classic senior opener. Knowing the flow shows you understand the framework, not just its surface API.

THE PATH OF A REQUEST

`public/index.php` → bootstrap the app → the HTTP Kernel runs global middleware → the router matches a route → route/controller middleware → the controller action runs → a Response flows back out through the middleware → sent to the client. Service providers are registered and booted during bootstrap, before routing.

MIDDLEWARE = LAYERS

Like an onion — each request passes in through the layers and the response passes back out. Auth, CORS, throttling, and session all live here. Great mental model to have ready.

```
public function
handle($request, Closure
$next)
{
    // before...
    $response =
    $next($request);
    // after...
    return $response;
}
```

SHARED-NOTHING

Each request boots a fresh app and tears it down after (unless you run Octane). So no in-memory state survives between requests — state lives in Redis / the database. This is why Laravel scales horizontally: add more PHP-FPM workers behind a load balancer.

Worth naming: Laravel Octane (Swoole/RoadRunner) keeps the app booted in memory between requests for big throughput gains — but you must avoid leaking state between requests. Good to mention as a modernisation lever.

3 · Service Container & DI

WHY IT MATTERS

DI is the backbone of a testable Laravel app, and modernising a legacy app almost always means moving logic out of fat controllers and into injected, single-responsibility services. Be fluent here.

Plain English: the container is Laravel's factory + registry. You declare what a class needs in its constructor; the container builds and injects those dependencies for you (automatic resolution). You program to interfaces, and bind the concrete implementation in one place.

BIND AN INTERFACE

```
// In a service provider's
register()
$this->app->bind(
    PaymentGateway::class,
    StripeGateway::class,
);

// singleton – one shared
instance
$this->app->singleton(
    SearchClient::class,
    fn($app) => new
    SearchClient(config('search.key')),
);
```

IT JUST INJECTS

```
class CheckoutController
{
    public function
    __construct(
        private
        PaymentGateway $gateway,
    ) {}
    // Laravel resolves
    StripeGateway
    // automatically – no
    `new`
}
```

bind vs singleton: `bind` resolves a fresh instance each time; `singleton` resolves once and reuses it. Contextual binding lets two classes get different implementations of the same interface. Method injection works too – type-hint a dependency in a controller action and Laravel injects it.

Say this: “I bind interfaces to implementations in a provider and let the container inject them — it gives me loose coupling and lets me swap a fake in tests without touching the class.”

4 · Service Providers & Facades

SERVICE PROVIDERS

The bootstrapping centre of the app. ``register()`` binds things into the container (no resolving other services yet); ``boot()`` runs after all providers are registered — the place for event listeners, route model bindings, view composers, macros.

```
class AppServiceProvider
extends ServiceProvider
{
    public function
    register(): void
    {
        $this->app-
        >singleton(Pricing::class);
    }
    public function
    boot(): void
    {
        Model::preventLazyLoading(!
        app()->isProduction());
    }
}
```

FACADES

A static-looking proxy to a service resolved from the container — ``Cache::get()`` is really ``app('cache')->get()`` under the hood. Convenient, and fully mockable in tests (``Cache::shouldReceive(...)``).

```
Cache::put('k', $v, 600);
DB::transaction(fn()
=> ...);
Log::info('done');

// The DI purist's
alternative:
public function
__construct(
    private
    CacheRepository $cache,
) {}
```

Talking point for modernisation: facades are quick but hide dependencies. In new code, constructor injection is easier to test and reason about; facades are fine for framework-level concerns (``Log``, ``Cache``, ``DB``). Knowing when to use which is the senior signal.

DEFERRED PROVIDERS

A provider can be deferred so its bindings load lazily only when needed — a small boot-time win across a large app with many providers.

5 · Routing, Middleware & Controllers

ROUTES & MODEL BINDING

```
Route::apiResource('orders',
OrderController::class);

Route::get('/orders/
{order}', function (Order
$order) {
    // route-model
    binding: {order} is
    // auto-resolved by id
    (404 if missing)
    return new
    OrderResource($order);
})-
>middleware('auth:sanctum');
```

MIDDLEWARE USES

- auth / sanctum — who are you
- throttle — rate limiting
(`throttle:60,1`)
- can — authorize a policy
- Custom: tenant scoping, request-id, audit logging

Keep controllers thin

A fat controller is the #1 legacy smell. Validate in a Form Request, do work in an injected service or action class, and return an API Resource. The controller just orchestrates.

✗ FAT CONTROLLER

```
public function
store(Request $r)
{
    $data = $r-
>validate([...]);
    // 60 lines of
    business logic,
    // DB writes, emails,
    API calls...
}
```

✓ THIN CONTROLLER

```
public function store(
    StoreOrderRequest
    $request,
    PlaceOrder
    $placeOrder,
    ) {
    $order = $placeOrder-
>handle(
        $request-
>validated()
    );
    return new
    OrderResource($order);
}
```

6 · Validation & Form Requests

Validation is your first line of defence and a place legacy apps get messy. Move rules into a Form Request so the controller stays clean and the rules are reusable and testable.

```
class StoreOrderRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('create', Order::class);
    }

    public function rules(): array
    {
        return [
            'items' => ['required', 'array', 'min:1'],
            'items.*.sku' => ['required', 'string',
                'exists:products,sku'],
            'items.*.qty' => ['required', 'integer', 'min:1'],
            'coupon' => ['nullable', 'string', new
                ValidCoupon],
            'shipping_country' => ['required', Rule::in(['GB',
                'IE'])],
        ];
    }
}
```

WHY IT'S BETTER

- `authorize()` gates the request before rules even run
- Rules live with the request, not scattered
- `$request->validated()` returns only clean data
- Custom rules (`ValidCoupon`) are unit-testable

WORTH NAMING

- Nested / array rules with dot notation
- `prepareForValidation()` to normalise input
- `Rule::unique()` ignoring self on update
- Validation runs before your logic — fail fast

7 · API Resources & REST in Laravel

WHY IT MATTERS

A marketplace lives and dies by its API. Eloquent API Resources are how you shape a consistent JSON contract and avoid leaking DB columns straight to the client.

A RESOURCE

```
class OrderResource
extends JsonResponse
{
    public function
toArray($request): array
    {
        return [
            'id' =>
$this->id,
            'total' =>
$this->total_pennies /
100,
            'status' =>
$this->status,
            'items' =>
ItemResource::collection(
                $this-
>whenLoaded('items')
            ),
        ];
    }
}
```

WHY whenLoaded()

Only includes the relationship if it was eager-loaded — so the resource never triggers a hidden N+1 (see §9). This one method is a senior tell.

```
// Controller decides what
to load
Order::with('items')
->paginate(20);
// →
OrderResource::collection(...)
```

REST conventions & versioning

Method / Route	Meaning	Status on success
GET /api/v1/orders	list (paginated)	200
POST /api/v1/orders	create	201 Created
PATCH /api/v1/orders/42	partial update	200
DELETE /api/v1/orders/42	delete	204 No Content
(validation fails)	unprocessable	422 + error bag
(rate limited)	too many requests	429

Always version (`/api/v1`) so you can ship breaking changes without breaking clients — critical for an app used by millions with existing integrations.

8 · Eloquent Relationships

WHY IT MATTERS

With 800+ models, relationships are the app. Fluency here — and knowing which relationship maps to which schema shape — is non-negotiable for this role.

THE COMMON TYPES

- `hasOne / belongsTo` — 1:1, FK on the child
- `hasMany` — 1:many
- `belongsToMany` — many:many via a pivot table
- `hasManyThrough` — reach across an intermediate
- `morphMany / morphTo` — polymorphic (comments on many types)

IN CODE

```
class User extends Model {
    public function
    orders() {
        return $this-
>hasMany(Order::class);
    }
    public function
    roles() {
        return $this-
>belongsToMany(Role::class)
        -
>withTimestamps()
        -
>withPivot('assigned_by');
    }
}
```

Polymorphic — the one they may probe

A `comments` table with `commentable_id` + `commentable_type` can attach to orders, products, users — anything. Powerful, but the polymorphic column can't have a normal foreign key, so integrity is enforced in code.

```
class Comment extends Model {
    public function commentable() { return $this->morphTo(); }
}
class Product extends Model {
    public function comments() { return $this-
>morphMany(Comment::class, 'commentable'); }
}
```

9 · The N+1 Problem

WHY IT MATTERS

This came up in their world for sure — it's the single most common performance killer in any Eloquent app, and it gets worse the more data you have. On a marketplace, an unnoticed N+1 can fire thousands of queries per page.

✗ N+1 — 1 + N QUERIES

```
$orders =
Order::all();           // 1
foreach ($orders as
$order) {
    echo $order->user-
>name; // +1 each
}
// 500 orders = 501
queries
```

✓ EAGER LOAD — 2 QUERIES

```
$orders =
Order::with('user')-
>get();
foreach ($orders as
$order) {
    echo $order->user-
>name;
}
// 2 queries, regardless
of count
```

The tools

- ``with()`` — eager load up front. Nested & constrained: ``with(['items.product', 'user.id,name'])``.
- ``load()`` — lazy-eager load on an existing model.
- ``withCount()`` — counts without loading the rows.
- ``Model::preventLazyLoading()`` in dev — throws the moment an N+1 happens, so you catch it before production.

```
// Only load what you need + a count, no N+1
$orders = Order::with(['items:id,order_id,product_id,qty'])
->withCount('items')
->where('status', 'paid')
->latest()
->paginate(20);
```

Say this: “I eager load relationships rather than lazy loading in a loop, and I turn on `preventLazyLoading` in dev so an N+1 fails loudly instead of silently firing a thousand queries in prod.”

10 · Scopes, Casts & Observers

QUERY SCOPES

Reusable query fragments. Global scopes apply to every query (e.g. soft-deletes, tenant scoping) — powerful and dangerous.

```
// local scope
public function
scopePaid($q) {
    return $q-
>where('status', 'paid');
}
Order::paid()->latest()-
>get();
```

CASTS

Cast raw DB columns to rich types automatically — no manual encode/decode.

```
protected function
casts(): array {
    return [
        'meta' =>
'array',
        'is_active' =>
'boolean',
        'settings' =>
AsEncrypted::class,
        'price' =>
MoneyCast::class,
    ];
}
```

Observers & model events

Hook into a model's lifecycle (`creating`, `updated`, `deleting`, ...) to keep side-effects out of controllers — audit logs, cache busting, denormalised counters. Keep them fast; push slow work to a queue.

```
class OrderObserver {
    public function created(Order $order): void {
        Cache::forget("user:{$order->user_id}:orders");
        SendOrderConfirmation::dispatch($order); // queue the slow
    }
}
```

⚠ GLOBAL SCOPE TRAP

A global tenant scope is great — until a raw query or a `withoutGlobalScopes()` call quietly bypasses it and leaks one customer's data to another. Know that trade-off before you lean on them.

Part Two · Data at Scale

The ad is emphatic about SQL / MySQL / MariaDB indexes & table design and big data. This is the half of the interview where a marketplace round goes deep — because at their scale, the database is the bottleneck. Own this part and you own the interview.

SCHEMA & INDEXES

- 11. Indexes & Table Design
- 12. EXPLAIN & Query Tuning
- 13. Migrations at Scale

VOLUME

- 14. Big Data in Laravel
- 15. Transactions & Locking
- 16. Caching & Redis

ASYNC & INTEGRATIONS

- 17. Queues & Horizon
- 18. Events & Listeners
- 19. Third-party APIs
- 20. Webhooks

11 · MySQL/MariaDB Indexes & Design

WHY IT MATTERS

They list this as a must-have, twice over. Be ready to explain how an index works, when it won't be used, and how you'd design a table that stays fast at millions of rows.

How an index actually works

InnoDB stores the table itself as a B-tree keyed on the primary key — the clustered index. A secondary index is a second B-tree whose leaves hold the PK, so a lookup by a secondary index is two hops: find the PK, then fetch the row. This is why a small PK (`BIGINT`, not a random UUID) matters — it sits inside every secondary index.

COMPOSITE & LEFTMOST-PREFIX

```
CREATE INDEX
idx_user_status
ON orders (user_id,
status);
-- helps: WHERE user_id=1
AND status='paid'
-- helps: WHERE user_id=1
-- NO help: WHERE
status='paid'
--          (no leftmost
prefix)
```

COVERING INDEX

If the index holds every column the query reads, MySQL answers from the index alone — no row fetch. EXPLAIN shows `Using index`.

```
CREATE INDEX idx_cover
ON orders (user_id,
status, total);
-- SELECT total ... WHERE
user_id=1
--   AND status='paid' →
index only
```

Column order: equality, then range, then sort

```
-- WHERE tenant_id=? AND status=? AND created_at > ? ORDER BY
created_at
CREATE INDEX idx ON orders (tenant_id, status, created_at);
--                      equality equality range + sort
-- Satisfies filter AND sort — no 'Using filesort'.
```

Design principles: right-size types (`INT` over `BIGINT` where it fits, `VARCHAR(50)` over `(255)`, `TIMESTAMP` over `DATETIME`); normalise for integrity, denormalise a counter for read speed; never index blindly — every index slows writes.

11 · When an Index Won't Be Used

Knowing these saves hours of head-scratching — and naming them cold is a strong senior signal.

```
-- ✗ function on the column → index ignored
WHERE YEAR(created_at) = 2024;
-- ✓ rewrite as a range so the index works
WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01';

-- ✗ leading wildcard → no index (use FULLTEXT for search)
WHERE name LIKE '%smith';

-- ✗ type mismatch → silent cast, index skipped
WHERE phone = 07911123456; -- phone is VARCHAR; quote it

-- ✗ low selectivity → optimiser prefers a scan
WHERE is_active = 1; -- if 95% of rows are active
```

FULLTEXT & SEARCH

`LIKE '%x%'` can't use a B-tree index. For real search, a FULLTEXT index, or an external engine (Meilisearch / Elasticsearch via Laravel Scout) on a marketplace catalogue.

SELECTIVITY

An index only helps if it narrows results a lot. A boolean flag where 95% of rows match isn't worth indexing on its own — but it can be a useful trailing column in a composite.

Say this: “Before adding an index I run EXPLAIN, check selectivity, and order composite columns equality-first. I reach for a covering index or a denormalised count before I reach for more hardware.”

12 · EXPLAIN & Query Tuning

EXPLAIN is the first move on any slow query — run it before you optimise anything.

```
EXPLAIN SELECT * FROM orders WHERE user_id = 42;
-- MySQL 8+ / MariaDB: actual timings, not estimates
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;
```

Reading it like a senior

Column	Want to see	Red flag
type	const, eq_ref, ref, range	ALL (full table scan)
key	the index you expect	NULL (no index used)
rows	small vs table size	millions
filtered	high %	low % = scan then discard
Extra	Using index	Using filesort / Using temporary

FIND THE OFFENDERS

```
SET GLOBAL slow_query_log
= 'ON';
SET GLOBAL long_query_time
= 1;
-- summarise, top 10 by
time
-- mysqldumpslow -s t -t
10 slow.log
```

IN LARAVEL, IN DEV

```
DB::enableQueryLog();
// ...run the page...
logger()->info('q', [
    'n' =>
    count(DB::getQueryLog())
]);
// Debugbar / Clockwork
show
// query count + timings
per request
```

Say this: “First thing I’d do is run EXPLAIN and check whether we’re doing a full table scan or a filesort — nine times out of ten the fix is an index or killing an N+1, not more hardware.”

13 · Migrations & Schema at Scale

WHY IT MATTERS

Their app has 800+ models. Schema change is a daily reality, and on a big, live table a careless migration is an outage. Show you think about migrations as production operations, not just local `php artisan migrate`.

A CLEAN MIGRATION

```
Schema::create('orders',
function (Blueprint $t) {
    $t-
    >id(); //
    BIGINT PK
    $t-
    >foreignId('user_id')
        ->constrained()
        ->cascadeOnDelete();
    $t->string('status')-
    >index();
    $t-
    >unsignedInteger('total_pennies');
    $t->timestamps();
    $t->index(['user_id',
    'status']); // composite
});
```

THE BIG-TABLE TRAP

Adding a column or index to a multi-million-row table can lock it and stall the app. Mitigations to name:

- Online DDL (`ALGORITHM=INPLACE`) where supported
- pt-online-schema-change / gh-ost for zero-downtime
- Deploy schema change and code change in separate, backward-compatible steps (expand → migrate → contract)

Practical rules

- Every migration is reversible — a working `down()` matters when a deploy goes wrong.
- Foreign keys enforce integrity; cascade deliberately.
- Add a column nullable / with a default first, backfill in a queued job, then make it required — never one giant blocking migration.
- Keep the schema dump (`schema:dump`) current so fresh installs don't replay hundreds of migrations.

14 · Big Data in Laravel

WHY IT MATTERS

“Worked with big data and enjoys it” is a stated must-have. The core skill is never loading more into memory than you need — and Laravel gives you clean tools for exactly that.

✗ LOADS EVERYTHING

```
// pulls every row into
memory,
// then hits memory_limit
and dies
foreach (Order::all() as
$o) {
    export($o);
}
```

✓ STREAM IT

```
// N rows at a time,
constant memory
Order::chunkById(1000,
function ($chunk) {
    foreach ($chunk as $o)
        export($o);
});

// or a lazy cursor
(generator underneath)
foreach (Order::lazy() as
$o) export($o);
```

The toolbox

- `chunkById()` over `chunk()` when you're also modifying rows (stable paging by PK, no skipped rows).
- `cursor()` / `lazy()` — a `LazyCollection` streaming one row at a time.
- `LazyCollection` — apply `map` / `filter` lazily over a huge source without materialising it.
- Bulk insert / upsert — `Model::upsert(\$rows, ['sku'], ['price'])` beats thousands of individual inserts.
- Queue the whole job (§17) and batch it so it runs off the request cycle.
- Always paginate list endpoints — `LIMIT/OFFSET`, or keyset (`WHERE id > ?`) for deep pages where OFFSET gets slow.

```
// Keyset pagination — scales where OFFSET doesn't
$page = Order::where('id', '>', $lastId)
    ->orderBy('id')
    ->limit(100)
    ->get();
```

15 · Transactions, ACID & Locking

WHY IT MATTERS

Money and inventory on a marketplace must not half-happen. Being able to describe a race condition and its fix is exactly the backend rigour they're hiring for.

ACID + SAFE WRITE

```
DB::transaction(function
() use ($from, $to) {
    $from-
>decrement('balance',
100);
    $to-
>increment('balance',
100);
    // any throw →
    automatic rollback
});
```

Atomic · Consistent · Isolated · Durable.
`DB::transaction()` commits on success, rolls back on any exception, and can auto-retry on deadlock.

ISOLATION LEVELS

- READ COMMITTED — non-repeatable reads (Postgres default)
- REPEATABLE READ — MySQL/InnoDB default
- SERIALIZABLE — safest, slowest

InnoDB's gap locking largely prevents phantoms even at REPEATABLE READ.

The classic race: last item in stock

✗ LOST UPDATE

```
$stock = $product-
>stock; // both read 1
if ($stock > 0)
{
    // both pass
    $product-
>decrement('stock');
}
// sold twice, stock = -1
```

✓ LOCK THE ROW

```
DB::transaction(function
() use ($id) {
    $p =
Product::whereKey($id)
->lockForUpdate() // 2nd
req waits
->first();
    if ($p->stock > 0) $p-
>decrement('stock');
});
```

Even simpler: push the check into the write — ``UPDATE products SET stock = stock - 1 WHERE id = ? AND stock > 0`` — then check affected rows. The database does the locking and stock can never go negative.

16 · Caching Strategies & Redis

WHY IT MATTERS

Data caching is a stated must-have. On a read-heavy marketplace, the right cache is the difference between the DB coping and the DB falling over.

THE LAYERS

- OPcache — compiled PHP bytecode (on in prod, always)
- App cache — Redis for data/results
- HTTP cache — `Cache-Control`, ETags, CDN for assets
- Query/computed results — expensive aggregates

CACHE-ASIDE (LARAVEL)

```
$user = Cache::remember(
    "user:$id", 3600,
    fn() =>
        User::findOrFail($id),
);
// bust on write:
Cache::forget("user:$id");
```

Beyond the basics

- Invalidation is the hard part — set a sensible TTL and bust the key on write. “There are only two hard things: cache invalidation and naming things.”
- Cache tags (Redis) group related keys so you can flush a whole set at once.
- `Cache::lock()` — a distributed lock to stop a thundering herd (many requests rebuilding the same expensive value on a cold cache).
- Redis also powers queues, sessions, and rate limiting — one dependency, several jobs.
- Cache IDs / small DTOs, not giant object graphs; watch memory and eviction policy (`allkeys-lru`).

```
Cache::lock("rebuild:home-feed", 10)->get(function () {
    return Cache::put('home-feed', $this->buildExpensiveFeed(), 300);
});
```

17 · Queues & Background Jobs

WHY IT MATTERS

Job queues are a stated must-have. Anything slow — emails, image resizing, calling Stripe/ OpenAI, exports — must not block the HTTP response. This is core backend signal.

A producer dispatches a job onto a broker (Redis / SQS); a separate worker process pulls and runs it asynchronously. In production you run Laravel Horizon to supervise Redis queues with dashboards and metrics.

```
// Producer – returns instantly, user isn't kept waiting
ProcessPayout::dispatch($order)->onQueue('payments');

// The job
class ProcessPayout implements ShouldQueue {
    public int $tries = 3;
    public array $backoff = [10, 30, 60]; // retry with backoff

    public function handle(PayoutService $svc): void {
        $svc->pay($this->orderId); // pass an ID, not the
        whole model
    }
}
```

GET RIGHT

- Idempotent jobs — safe to run twice
- Retries with backoff; a failed_jobs table / dead-letter for poison jobs
- Pass IDs, not whole objects (models re-fetch fresh)
- Batching & chaining for multi-step workflows

BATCHING

```
Bus::batch([
    new ImportChunk(1),
    new ImportChunk(2),
])->then(fn() =>
    Notify::dispatch()
        ->onQueue('imports')
        ->dispatch());
```

Say this: “I push slow or third-party work to a queue and keep jobs idempotent with retry-and-backoff, so a flaky external API becomes a retry, not a failed request. Horizon gives me visibility over throughput and failures.”

18 · Events, Listeners & Observers

Events let one thing happen and many things react — without coupling them together. Perfect for the “when an order is placed, do X, Y and Z” problems a marketplace is full of.

```
// The order just announces what happened
OrderPlaced::dispatch($order);

// Listeners subscribe independently (queue the slow ones)
class SendReceipt implements ShouldQueue { public function
handle(OrderPlaced $e) {...} }
class ReserveStock { public function
handle(OrderPlaced $e) {...} }
class NotifySeller implements ShouldQueue { public function
handle(OrderPlaced $e) {...} }
```

WHY DECOUPLE

Adding a new reaction (analytics, fraud check) means adding a listener, not editing the checkout code. Open/closed in practice — and each listener is independently testable and queueable.

vs OBSERVERS vs JOBS

- Observer — tied to model lifecycle (`created`, `deleted`)
- Event/Listener — domain events, many reactions
- Job — a single unit of deferred work

They compose: an event's listener dispatches a job.

DON'T OVERDO IT

Events add indirection. For a simple, single side-effect, a direct call in a service is clearer. Reach for events when there are genuinely several independent reactions — say that trade-off and you sound senior.

Part Three · APIs, Ops & Modernising

They integrate with Stripe, Google, AWS and OpenAI, run at marketplace scale, and the job itself is to modernise their Laravel app. This part is the integration, testing, security and DevOps layer — plus how to talk about improving a large legacy codebase without breaking it.

INTEGRATIONS

- 19. Third-party APIs
- 20. Webhooks & Idempotency
- 21. Microservices

QUALITY & SECURITY

- 22. Testing Laravel
- 23. Security

RUNNING IT

- 24. DevOps & Backups
- 25. Modernising a Legacy App
- 26. The Problem-Solver

19 · Third-party APIs

WHY IT MATTERS

“Worked with many APIs, e.g. Stripe, Google, AWS, OpenAI” is a must-have. Show you integrate robustly — timeouts, retries, wrapping the vendor behind your own interface — not just calling a URL and hoping.

```
use Illuminate\Support\Facades\Http;

$response = Http::withToken(config('services.openai.key'))
    ->timeout(15)
    ->retry(3, 200, throw: false)           // 3 tries, 200ms apart
    ->post('https://api.openai.com/v1/chat/completions', [
        'model' => 'gpt-...',
        'messages' => $messages,
    ]);

if ($response->failed()) {
    Log::warning('OpenAI call failed', ['status' => $response-
    >status()]);
    return $this->fallback();                // degrade gracefully
}
```

WRAP THE VENDOR (ADAPTER)

Hide the SDK behind your interface so a vendor change or price hike stays contained in one class — and you can fake it in tests. This is the Adapter pattern, and it’s exactly the mindset behind their “Google Maps raised fees 100x, find a workaround” question (§26).

```
interface Geocoder {
    public function
    locate(string $q): LatLng;
}
class GoogleGeocoder
implements Geocoder {}
class MapboxGeocoder
implements Geocoder {}
```

ROBUSTNESS CHECKLIST

- Timeouts on every call — never hang a request
- Retry with backoff on transient failures
- Circuit breaker / fallback when a provider is down
- Do it in a queued job so a slow API doesn’t block the user
- Cache responses where allowed (geocoding, rates)
- Keys in `.env` / config`, never in code

20 · Webhooks & Idempotency

WHY IT MATTERS

Stripe (and most payment/marketplace flows) push events to you via webhooks. Handling them safely — verifying signatures and processing exactly once — is a very common senior interview probe.

```
public function handle(Request $request)
{
    // 1. VERIFY the signature — never trust the payload blindly
    $event = Webhook::constructEvent(
        $request->getContent(),
        $request->header('Stripe-Signature'),
        config('services.stripe.webhook_secret'),
    );

    // 2. IDEMPOTENCY — Stripe may deliver the same event twice
    if (ProcessedEvent::where('event_id', $event->id)->exists()) {
        return response()->noContent(); // already handled
    }

    // 3. Do the minimum inline, queue the rest
    ProcessedEvent::create(['event_id' => $event->id]);
    HandleStripeEvent::dispatch($event->id);

    // 4. Return 2xx FAST so the provider stops retrying
    return response()->noContent();
}
```

THE FOUR RULES

1. Verify the signature (HMAC) before anything
2. Be idempotent — store the event id, ignore duplicates
3. Return 2xx quickly — offload work to a queue
4. Expect retries & out-of-order delivery

IDEMPOTENCY KEYS (OUTBOUND)

The mirror image: when you call Stripe to charge, send an Idempotency-Key so a retry after a timeout doesn't double-charge the customer. Same principle, both directions.

21 · Microservices & SOA

WHY IT MATTERS

Listed under “would be lovely”. You don’t need to have run a fleet — but know the trade-offs and, crucially, be honest about when a modular monolith is the better call. That nuance is the senior answer.

WHAT & WHY

Independent services, each owning its data, talking over HTTP/queues.
Benefits: independent deploy & scaling, team autonomy, fault isolation. A payments service can scale separately from search.

THE REAL COSTS

- Distributed transactions are hard (no cross-service ACID → sagas)
- Network latency & partial failure everywhere
- Operational overhead — observability, deploys, versioning
- Data consistency becomes eventual

The pragmatic view

Most teams are better served by a well-structured modular monolith first — clear domain boundaries, a clean service layer — then extract a service only when a real scaling or team boundary demands it. Communication between services leans on queues/events (async, resilient) over synchronous chains where possible.

Say this: “I’d extract a service when there’s a genuine scaling or ownership boundary — not by default. Premature microservices trade one hard problem for a dozen distributed ones. A modular monolith with clear boundaries gets you most of the benefit at a fraction of the cost.”

22 · Testing Laravel

WHY IT MATTERS

Modernising a legacy app safely is impossible without tests — they're the safety net that lets you refactor with confidence. Expect to be asked how you'd add coverage to code that has none.

THE PYRAMID

Many fast unit tests, fewer feature/integration tests (real DB), fewest end-to-end. Laravel's feature tests hit real routes through the framework — brilliant value for confidence per line.

THE TOOLS

- Pest or PHPUnit
- Factories — generate model data
- RefreshDatabase — clean DB per test
- Http::fake() — stub external APIs
- Queue::fake() / Event::fake() — assert dispatch
- Bus::fake(), Mail::fake(), `travel()` for time

```
it('places an order and charges the customer', function () {
    Http::fake(['api.stripe.com/*' => Http::response(['id' => 'ch_1'],
    200)]);
    Queue::fake();

    $user = User::factory()->create();
    $product = Product::factory()->create(['stock' => 5]);

    $this->actingAs($user)
        ->postJson('/api/v1/orders', ['items' => [['sku' => $product-
>sku, 'qty' => 1]])
        ->assertCreated();

    expect($product->fresh()->stock)->toBe(4);
    Queue::assertPushed(SendReceipt::class);
});
```

Say this: “On a legacy app I start with feature tests around the critical money paths — they pin current behaviour so I can refactor underneath without fear. If something's hard to test, that's usually the design telling me to inject a dependency.”

23 · Security in Laravel

Laravel handles a lot for you — but knowing what it protects against, and where the gaps are, matters on an app handling millions of users' data and payments.

MOSTLY HANDLED — KNOW WHY

- SQL injection — the query builder / Eloquent bind parameters (don't concatenate into `DB::raw`)
- XSS — Blade `{!!}` escapes output (`{!! !!}` does not)
- CSRF — the `@csrf` token on web forms
- Passwords — `Hash::make()` (bcrypt/argon), never plain/MD5

YOUR RESPONSIBILITY

- Mass assignment — set `fillable` / `guarded` deliberately
- Authorization — Policies & Gates, not just authentication
- Rate limiting on auth and API routes
- Secrets in `.env`, never committed; `.env` in `.gitignore`
- `APP\_DEBUG=false` in prod — a stack trace is a data leak

```
// SAFE — bound parameter, even in a raw fragment
User::whereRaw('email = ?', [$email])->first();

// Authorization, not just auth
$this->authorize('update', $order); // runs OrderPolicy@update
```

MULTI-TENANCY / DATA ISOLATION

On a marketplace, the classic breach is one seller seeing another's data. Scope every query by the owner — a global scope or policy makes correct the default, not something each developer must remember.

24 · DevOps & Backups

WHY IT MATTERS

Listed as “would be lovely” — they even name “taking a MySQL backup”. You don’t need to be an SRE; show you can deploy safely and wouldn’t be helpless if the database needed restoring.

MYSQL BACKUP & RESTORE

```
# logical dump (portable)
mysqldump -u user -p --
single-transaction \
  --routines myapp >
backup.sql

# restore
mysql -u user -p myapp <
backup.sql

# large DBs: physical/
binary backup
# (Percona XtraBackup) +
binlog for PITR
```

`--single-transaction` gives a consistent snapshot without locking InnoDB tables. Point-in-time recovery replays the binlog on top of the last full backup.

DEPLOY & RUN

- Docker for parity: same image local → prod
- CI runs tests before merge; zero-downtime deploy (Envoyer / Deployer / rolling)
- On deploy: `migrate --force`, clear OPcache, `config:cache`, restart queue workers
- 12-factor config — differences live in `.env`, never in code
- Backups are only real if you’ve tested a restore

OBSERVABILITY

You can’t step-debug prod. Lean on structured logs (with a request id), metrics (error rate, p95 latency), and APM traces (New Relic / Datadog / Sentry). In an incident: roll back first, diagnose from the signals second.

25 · Modernising a Legacy Laravel App

WHY IT MATTERS

This is the job — “assist in modernising the Laravel app”. Have a clear, safe, incremental method. The instinct they’re screening against is the big-bang rewrite.

THE METHOD — STRANGLE, DON’T REWRITE

1. Measure & map — where’s the pain (slow pages, error rates, fat classes)? Don’t guess.
2. Get a safety net — feature tests around the critical paths first.
3. Upgrade the foundation — get onto a supported PHP & Laravel version (Rector automates much of it); free performance and security.
4. Refactor incrementally — fat controllers → services; magic strings → enums; N+1s → eager loads; add indexes where EXPLAIN says so.
5. Strangle — build new features cleanly alongside the old, migrate traffic gradually, delete the old path once nothing uses it.

QUICK WINS TO NAME

- Turn on OPcache, add missing indexes
- Kill the worst N+1s (Debugbar finds them)
- Move slow work to queues
- Add caching to the hottest read paths
- Upgrade PHP — often the cheapest speedup there is

WHY NOT A REWRITE

A full rewrite means months with no new value, high risk, and a real chance of reproducing the same bugs. Incremental modernisation ships value continuously and keeps the business running — exactly what a “fast-growing” company needs.

26 · The Problem-Solver Mindset

WHY IT MATTERS

They explicitly want someone who solves problems rather than works tickets — and gave the exact example: “Google Maps increased fees by 100x, find a workaround.” Have a structured way to answer this class of question.

A FRAMEWORK FOR THE ‘FIND A WORKAROUND’ QUESTION

1. Clarify the real constraint — is it cost, a hard rate limit, an outage? What’s the actual budget / SLA?
2. Quantify — how many calls, how often, how much are we really spending? Measure before acting.
3. Reduce demand — cache results, batch requests, debounce, precompute. Often 90% of calls are avoidable.
4. Find alternatives — is there a cheaper provider (Mapbox, OpenStreetMap / Nominatim), a self-hosted option, or a good-enough offline dataset?
5. Abstract the vendor — because you wrapped it behind an interface (\$19), swapping providers is a config change, not a rewrite.
6. Trade-offs, out loud — cost vs accuracy vs effort. Recommend one, and say why.

WHAT THEY’RE LISTENING FOR

Not a memorised answer — a thinking process: clarify → measure → reduce → alternatives → trade-offs. Curiosity, pragmatism, and cost-awareness. Exactly the traits the ad keeps repeating.

THE ONE-LINER

First I’d measure what we’re actually spending and why, then attack demand with caching and batching before I even look at alternatives — and because the provider sits behind our own interface, switching is a config change, not a rewrite.

Quick Reference · Phrases to Use

Drop these into answers — they signal you think about why, not just how.

Topic	Key phrase
N+1	Eager load relationships rather than lazy loading in a loop; preventLazyLoading in dev.
Indexes	Covering & composite, equality-first — verify with EXPLAIN before adding.
Slow query	First move is EXPLAIN — check for a full scan or filesort.
Big data	Chunk or cursor; never load a million rows into memory. Keyset over deep OFFSET.
Transactions	Wrap related writes so they all commit or all roll back — lockForUpdate for races.
Caching	Cache-aside with a TTL; the hard part is invalidation on write.
Queues	Push slow/third-party work to a queue; keep jobs idempotent with backoff.
Webhooks	Verify the signature, be idempotent, return 2xx fast, queue the rest.
APIs	Timeouts + retries, wrap the vendor behind an interface, degrade gracefully.
DI	Program to interfaces; the container injects — loose coupling and testability.
Thin controllers	Validate in a form request, do work in a service, return a resource.
Microservices	Extract on a real boundary, not by default — a modular monolith first.
Modernising	Strangler, not rewrite — safety net, upgrade the base, refactor incrementally.
Testing	Feature-test the money paths first; hard to test usually means bad design.
Multi-tenancy	Scope every query by owner by default, not by memory.

ASK THEM (SHOWS INTEREST)

- How big is the largest table, and where does it hurt today?
- What does the modernisation roadmap look like — versions, tech debt?
- Monolith or services now, and where do you want to be?
- How do you handle deploys, backups and incidents?
- What does success in the first 90 days look like?

You've got this, Chris

You build and ship real production systems and can explain your thinking clearly — that's most of the battle. Read the Why it matters boxes once more, tie every answer back to their marketplace, and go have a conversation, not an exam.